

TripWiz - Smart Travel Planning That Adapts to Reality

Team Name: TripWiz - Ariel University

Submitted by: David Kitinberg, Amit Bitton, Sagi Hassid

Executive Summary (Elevator Pitch)

Currently, planning a vacation is often done using clumsy Word documents or spreadsheets. It is time-consuming and frequently ruined at the last minute by unpredictable weather or poor time management. TripWiz is a Web platform that provides a convenient, visual workspace for organizing trips by days and hours, turning any bucket list into a realistic, fail-safe itinerary. The system cross-references attraction locations with real-time weather forecasts and operating hours, alerting you to any potential issues. Our goal is to provide travelers with an easy, intuitive planning experience, saving hours of work and ensuring the vacation goes exactly as planned.

Key Features

- **Visual Itinerary Builder:** Replaces clunky Word documents with a user-friendly, interactive interface that clearly displays the trip broken down by days and hours. *Customer Value:* A seamless planning experience that allows users to visually organize and edit all trip details in one place.
 - **Real-Time Collaborative Editing:** Allows multiple users to plan and adjust the itinerary simultaneously, functioning just like Google Docs.
Customer Value: Eliminates endless group chats by enabling friends or families to build their perfect trip together in one shared, live workspace.
 - **Smart Validation (Weather & Time):** Automatically checks the forecasted weather and operating hours for each planned attraction, alerting the user to any mismatches.
Customer Value: Prevents the frustration of arriving at a closed site or during a storm, ensuring a safe and optimized day.
 - **Dynamic Routing & Mapping:** Visually plots all waypoints (attractions, restaurants, hotels) on a dynamic map and calculates a logical route.
Customer Value: Easy spatial orientation and full control over daily navigation from a single, unified screen.
 - **Smart Fallback Recommendations:** Detects itinerary conflicts (like rain during a planned nature hike) and offers relevant, one-click alternatives nearby (like an indoor museum).
Customer Value: Acts as a "safety net" that turns any unexpected setback into a new opportunity, saving the vacation day.
-

Technical Implementation Plan

1. Technical Implementation Strategy

We propose a robust, scalable, and cost-efficient Serverless Architecture hosted entirely on AWS. This approach allows our team to focus purely on the application's core logic and user experience without managing underlying infrastructure, which is crucial for delivering a high-quality Proof of Concept (POC) within the required 4-5 week timeframe. By leveraging serverless compute, event-driven integrations, and managed databases, TripWiz will be able to handle asynchronous tasks like weather validation and API integrations seamlessly.

2. Core Technologies & AWS Services

To achieve the desired "smart" functionality, visual routing, and real-time collaboration, we have selected the following technology stack:

- **Frontend & Hosting:** The web application will be built using a modern JavaScript framework (e.g., React or Vue.js) to ensure a highly responsive UI. Hosted using **AWS Amplify**, providing out-of-the-box CI/CD capabilities.
- **Authentication & Security:** User identity management will be handled by **Amazon Cognito** to securely segment personal trip data.
- **API Layer (The "Front Door"):** **Amazon API Gateway** will act as the unified entry point for all frontend requests, managing both REST calls and active WebSocket connections.
- **Business Logic (Compute):** The core intelligence of TripWiz will reside in **AWS Lambda** functions, executing stateless logic on-demand.
- **Database & Storage:** **Amazon DynamoDB**, a fast and flexible NoSQL database, will store trip itineraries and waypoints in a highly scalable JSON format.

2.1. Feature-Specific Cloud Architecture

To support our advanced key features, we will implement the following specific AWS architectural patterns:

- **Real-Time Collaborative Editing:** Powered by **Amazon API Gateway (WebSocket APIs)** to maintain persistent, two-way communication between users, allowing itinerary changes to reflect instantly across all connected clients.
- **Smart Validation (Weather & Time):** Utilizes **Amazon EventBridge** to schedule automated, asynchronous checks. It triggers **AWS Lambda** to validate upcoming trip segments against weather APIs, and uses **Amazon SNS (Simple Notification Service)** to push immediate alerts to the user if a conflict is detected.
- **Dynamic Routing & Mapping:** Managed via **Amazon API Gateway** and **AWS Lambda** acting as a secure proxy. API keys for third-party mapping services are safely retrieved from **AWS Secrets Manager** during runtime, ensuring sensitive credentials are never exposed in the frontend code.
- **Smart Fallback Recommendations:** Leverages **Amazon Location Service** to perform rapid spatial queries (e.g., finding nearby indoor attractions within a specific radius) when the primary itinerary encounters a conflict.

3. External Integrations

- **Mapping & Routing:** Google Maps API (or an alternative like Mapbox) for visual route plotting and travel time calculations.
- **Weather & Environmental Data:** OpenWeatherMap API (or similar) to fetch real-time forecasts based on specific geographical coordinates and time slots.

4. Architectural Diagram Outline

1. **User Browser** makes HTTPS requests to **AWS Amplify** (Hosting).
2. Auth requests are routed to **Amazon Cognito**.
3. Application API calls (REST and WebSockets) are sent to **Amazon API Gateway**.
4. API Gateway triggers **AWS Lambda** functions for business logic.
5. Lambda retrieves secure keys from **AWS Secrets Manager**, fetches data from **External APIs** and **Amazon Location Service**, and reads/writes trip data to **Amazon DynamoDB**.
6. **Amazon EventBridge** triggers background Lambda checks, sending alerts via **Amazon SNS**.

[Link to the Diagram](#)

TripWiz: Serverless Architecture Deep-Dive

The proposed architecture for TripWiz leverages a fully managed, event-driven AWS Serverless paradigm. This design decouples the frontend from the backend, ensures infinite scalability, and relies heavily on stateless compute and managed services.

Below is the step-by-step breakdown of the data flow, security model, and business logic execution.

Phase 1: Content Delivery and Initialization

- **Component:** AWS Amplify & User Browser
- **The Flow:** The browser sends an HTTPS request to AWS Amplify, which acts as a CDN to deliver the compiled frontend assets. Once loaded, the User Browser acts as an autonomous client communicating directly with backend services.

Phase 2: Authentication and Security

- **Component:** Amazon Cognito
- **The Flow:** The client application sends authentication requests to Amazon Cognito. Upon successful verification, Cognito returns secure JWT (JSON Web Tokens) to the browser. These tokens are attached to future requests to authenticate API calls.

Phase 3: API Routing & Real-Time Collaboration

- **Component:** Amazon API Gateway (REST & WebSockets)
- **The Flow:** Standard actions use REST APIs. However, for **Real-Time Collaborative Editing**, the browser establishes a persistent connection via **Amazon API Gateway (WebSocket APIs)**. When one user updates an itinerary, the payload is sent through the WebSocket to a Lambda function, which updates the database and instantly broadcasts the change back through the API Gateway to all other active users viewing that trip.

Phase 4: Core Logic & Secure Proxies (Routing)

- **Component:** AWS Lambda & AWS Secrets Manager
- **The Flow:** For features like **Dynamic Routing & Mapping**, the frontend requests a route calculation. To protect third-party API keys, API Gateway triggers a Lambda function. This function securely fetches the required API key from **AWS Secrets Manager**, makes the external request to the mapping service, processes the data, and returns a clean JSON response to the user.

Phase 5: Spatial Intelligence & Fallbacks

- **Components:** Amazon Location Service & OpenWeather API
- **The Flow:** During planning, Lambda cross-references coordinates with the OpenWeather API. If a conflict is detected (e.g., rain at an outdoor park), the Lambda function queries **Amazon Location Service** to search for nearby indoor alternatives, providing instant **Smart Fallback Recommendations**.

Phase 6: Asynchronous Smart Validation

- **Components:** Amazon EventBridge & Amazon SNS
- **The Flow:** To ensure plans remain viable as the trip approaches, **Amazon EventBridge** triggers a scheduled Lambda function every night. This function checks the next day's itinerary against updated weather forecasts. If a sudden storm is predicted, the function triggers **Amazon SNS** to dispatch an email or SMS alert directly to the users.

Phase 7: Data Persistence

- **Component:** Amazon DynamoDB
- **The Flow:** All trip data is stored as complex, nested JSON objects in DynamoDB. This NoSQL structure allows rapid read/write operations of flexible schemas based on a Partition Key (UserID) and a Sort Key (TripID).

This architecture is highly decoupled, securely manages credentials, supports real-time multi-user editing, and provides proactive asynchronous alerts—making it a highly professional blueprint for a modern SaaS application.